

Comandos y funciones comunes de PHP

Este documento recoge comandos de línea de comandos de PHP (CLI) y una selección de funciones y construcciones comunes del lenguaje PHP, con ejemplos prácticos en cada caso. Está pensado como referencia rápida en español.

1. Comandos de la línea de comandos (CLI) de PHP

php -v

Muestra la versión de PHP instalada.

Ejemplo:

```
php -v
```

php -m

Lista las extensiones cargadas.

Ejemplo:

```
php -m
```

php -r '...'

Ejecuta código PHP directamente desde la línea de comandos (sin etiquetas `<?php ?>`).

Ejemplo:

```
php -r "echo 'Hola';"
```

php -a

Modo interactivo (interactive shell, REPL).

Ejemplo:

```
php -a
```

php -S host:puerto -t ruta

Levanta un servidor web embebido (útil para desarrollo).

Ejemplo:

```
php -S 127.0.0.1:8000 -t public/
```

php -d nombre=valor script.php

Define una directiva INI temporal para la ejecución.

Ejemplo:

```
php -d memory_limit=512M script.php
```

php -l archivo.php

Comprueba la sintaxis (lint) del archivo PHP.

Ejemplo:

```
php -l index.php
```

php -i

Muestra información de configuración (similar a `phpinfo()`).

Ejemplo:

```
php -i
```

2. Sintaxis básica y constructores

Etiquetas PHP

```
<?php
// Código aquí
?>
```

echo / print

Salida de texto.

Ejemplo:

```
echo 'Hola mundo';
print 'Hola';
```

Variables

```
$nombre = 'María';
$edad = 30;
```

Arrays

```
$arr = [1, 2, 3];
$assoc = ['clave' => 'valor'];
```

Funciones

```
function saludar($nombre) {
    return "Hola $nombre";
}
```

Clases - POO

```
class Persona {
    public $nombre;
    public function __construct($n) { $this->nombre = $n; }
    public function saludar() { echo "Hola {$this->nombre}"; }
}
$p = new Persona('Ana');
$p->saludar();
```

Try / Catch

```
try {
    // código
} catch (\Exception $e) {
```

```
    echo $e->getMessage();  
}
```

3. Funciones y comandos útiles (selección)

var_dump()

Muestra información sobre una variable (tipo y valor).

Ejemplo:

```
var_dump(\ $variable);
```

print_r()

Imprime estructuras (arrays/objetos) de forma legible.

Ejemplo:

```
print_r(\ $array);
```

isset() / empty()

Comprueban existencia y vacío.

Ejemplo:

```
if (isset(\ $x)) { }
```

```
if (empty(\ $x)) { }
```

include / require / _once

Incluyen archivos PHP.

```
require 'config.php';
```

```
include_once 'helpers.php';
```

strlen(), strpos(), substr()

Funciones de cadenas.

Ejemplo:

```
strlen('hola');
```

```
$pos = strpos('hola mundo', 'mundo');
```

```
$sub = substr('abcdef', 1, 3);
```

json_encode() / json_decode()

Convertir a/desde JSON.

```
\ $json = json_encode(\ $arr);
```

```
\ $arr = json_decode(\ $json, true);
```

file_get_contents() / file_put_contents()

Leer/escribir archivos rápidamente.

```
\ $texto = file_get_contents('fichero.txt');
```

```
file_put_contents('salida.txt', \ $texto);
```

fopen / fread / fclose

Operaciones de archivo de bajo nivel.

```
\ $f = fopen('fichero.txt', 'r');
```

```
\$data = fread(\$f, filesize('fichero.txt'));  
fclose(\$f);
```

header()

Enviar cabeceras HTTP (debe hacerse antes de salida).

Ejemplo:

```
header('Content-Type: application/json');
```

exec(), shell_exec(), passthru()

Ejecutar comandos del sistema (cuidado con seguridad/escapado).

```
\$out = shell_exec('ls -la');
```

PDO (conexión a BD)

Conectar a bases de datos de forma segura con PDO.

```
\$pdo = new PDO('mysql:host=localhost;dbname=test', 'user', 'pass');
```

password_hash() / password_verify()

Funciones seguras para almacenar/verificar contraseñas.

```
\$hash = password_hash(\$pass, PASSWORD_DEFAULT);
```

```
password_verify(\$pass, \$hash);
```

filter_var()

Validación y filtrado de datos.

```
\$email = filter_var(\$ _POST['email'], FILTER_VALIDATE_EMAIL);
```

session_start()

Iniciar sesión en PHP.

```
session_start();
```

```
\$ _SESSION['user'] = \$userId;
```

setcookie()

Crear cookies.

```
setcookie('nombre', 'valor', time()+3600, '/');
```

strtotime(), date()

Funciones de fecha/hora.

```
\$t = strtotime('2025-11-17');
```

```
\$hoy = date('Y-m-d H:i:s');
```

4. Buenas prácticas rápidas

- Usar PDO con sentencias preparadas para evitar SQL injection.
- Validar y sanear toda entrada del usuario (filter_var, filtros, escapes).
- No ejecutar comandos del sistema con datos no escapados.
- Manejar errores y excepciones; no mostrar trazas en producción.

- Configurar correctamente `display_errors` y `error_reporting` según entorno.
- Usar Composer para gestión de dependencias.

Generado por asistente — referencia rápida de comandos y funciones PHP.